

Implementazione del Gioco UNO

Francesco Bartolucci – Matricola 2274957

Anno Accademico 2025/2026

1 Descrizione generale del progetto

1.1 Introduzione

Questo progetto consiste nello sviluppo del gioco di carte **UNO**, realizzato in **Java** (Standard Edition 21/25). L'applicazione offre un'interfaccia grafica interattiva sviluppata con le librerie **Java Swing** e **AWT**.

Il gioco riproduce fedelmente le regole ufficiali del gioco da tavolo classico:

- Un mazzo regolamentare di **108 carte**, suddivise tra carte numeriche (da 0 a 9 nei quattro colori Rosso, Giallo, Verde e Blu), carte azione speciali (*Skip*, *Reverse*, *Draw Two*) e carte jolly (*Wild* e *Wild Draw Four*);
- Supporto per partite da 2 a 6 giocatori, con gestione dinamica dei turni in senso orario e antiorario;
- Applicazione delle penalità, meccaniche per dichiarare o contestare **UNO**, e regola del **Challenge** per la carta *Wild Draw Four* (+4);
- Calcolo ufficiale dei punteggi a fine round in base alle carte rimaste nelle mani degli avversari.

1.2 Struttura generale dell'applicazione

L'applicazione è organizzata separando la logica di gioco dall'interfaccia grafica in due package principali:

1. **backend**: contiene il modello dei dati (carte, colori, valori e mazzo), la gerarchia dei giocatori e le regole di gioco;
2. **gui**: gestisce l'interfaccia grafica Swing, le icone delle carte, il font personalizzato *Fredoka*, e il controller principale (**FinestraPrincipale**).

1.3 Funzionalità implementate

Il programma include le seguenti funzionalità:

- **Configurazione Iniziale**: una finestra di dialogo all'avvio per impostare numero di giocatori, nomi, ruoli (giocatore umano, *Bot Casuale* o *Bot Aggressivo*), modalità di gioco e varianti attive.
- **Due Modalità di Partita**:

- *Partita Singola*: composta da un unico round in cui vince il primo giocatore che esaurisce le carte in mano;
- *Partita a Punti*: strutturata su più round con soglia di vittoria personalizzabile (predefinita a 500 punti), con classifica e riepilogo a fine round.
- **Bot**: include un bot con scelte casuali (**BotCasuale**) e un bot strategico (**BotAggressivo**) che ostacola attivamente gli avversari vicini alla vittoria.
- **Regole Alternative**: possibilità di abilitare varianti opzionali come lo *Stacking* (accumulo di +2 e +4), il *Number Rush* (scarto cumulativo di carte con lo stesso valore) e il *Seven-Zero* (rotazione delle mani con lo 0 e scambio carte con il 7).

Il progetto non richiede dipendenze esterne ed è eseguibile su qualsiasi ambiente provvisto di Java Runtime Environment (JRE).

2 Modalità di gioco e gestione dei punteggi

Il sistema permette di scegliere tra due modalità di gioco principali.

2.1 Partita Singola

La *Partita Singola* si articola in un **singolo round**:

- **Obiettivo**: Vince il primo giocatore che riesce a scartare tutte le carte in proprio possesso.
- **Conclusione**: Al momento della vittoria, la partita termina e viene mostrato a schermo un messaggio di proclamazione.

2.2 Partita a Punti

La *Partita a Punti* si sviluppa su **più round consecutivi**:

- **Soglia di vittoria**: Il valore predefinito è **500 punti**, configurabile all’inizio.
- **Assegnazione punti a fine round**: Al termine di ogni round, il vincitore ottiene un punteggio pari alla somma dei punti delle carte rimaste in mano agli avversari.
- **Schermata di riepilogo**: Al termine del round si apre la finestra **SchermataPunteggi**, che mostra la classifica generale aggiornata.
- **Proseguimento o vittoria**:
 - Se nessun partecipante ha raggiunto la soglia, premendo *"Inizia Prossimo Round"* viene rimescolato il mazzo e avviato il round successivo.
 - Se un giocatore raggiunge o supera la soglia, la partita si conclude proclamando il vincitore finale.

Tabella 1: Valore in punti delle carte di UNO.

Tipologia Carta	Valori / Simboli	Punti
Carte Numeriche	Da ZERO a NOVE (0 – 9)	Valore nominale (0 – 9 pt)
Carte Azione	SKIP, REVERSE, DRAW_TWO (+2)	20 pt
Carte Jolly	WILD, WILD_DRAW_FOUR (+4)	50 pt

2.3 Tabella dei punteggi e calcolo

Il conteggio dei punti al termine di ciascun round rispetta la tabella ufficiale:

Nel codice, la classe `Giocatore` espone il metodo `calcolaPuntiMano()` che calcola la somma dei punti iterando sulle carte della mano, senza alterare lo stato della partita.

3 Scelte implementative e gestione delle regole

3.1 Gestione della carta Reverse a 2 giocatori

Nelle partite a due giocatori, il regolamento ufficiale stabilisce che la carta `REVERSE` agisca come un *Salta Turno*: chi la gioca scavalca l'avversario e può scartare subito un'altra carta.

Nelle partite standard con più giocatori, la gestione del verso inverte semplicemente il segno della variabile `direzione` (`direzione *= -1`), modificando l'avanzamento dei turni calcolato tramite aritmetica modulare:

```
indiceProssimo = (indiceCorrente + direzione + N) % N
```

Con due soli partecipanti ($N = 2$), la sola inversione del segno non basterebbe, dato che in modulo 2 muoversi di $+1$ o -1 porta sempre all'avversario. Per applicare correttamente la regola, quando $N = 2$ il metodo `applicaEffettiRegole()` esegue una chiamata aggiuntiva a `passaAlProssimoGiocatore()`: sommata al normale avanzamento di fine turno, questo doppio passaggio riporta il turno al giocatore attivo, riproducendo alla perfezione l'effetto dello `SKIP`.

3.2 Terminazione con carta speciale come ultima carta

Per evitare l'apertura di finestre di dialogo superflue quando l'ultima carta giocata per chiudere la mano è una carta ad effetto, è stato introdotto un controllo di terminazione immediato:

```
if (corrente.getNumeroCarte() == 0) return;
```

In questo modo la partita passa direttamente al conteggio dei punti e alla proclamazione della vittoria.

3.3 Dichiarazione e contestazione di UNO

Chi rimane con una sola carta in mano deve dichiarare *"UNO!"*. La gestione include:

1. Dichiarazione preventiva:

- Quando il giocatore ha esattamente **2 carte in mano** e si appresta a giocarne una, può premere il pulsante *"DICHARA UNO!"*;
- Il sistema registra l'avvenuta dichiarazione impostando `haDichiaratoUnoSuccesso = true`;
- Se il giocatore pesca una carta giocabile rimanendo con 2 carte, il sistema non la gioca automaticamente, lasciando il tempo di effettuare la dichiarazione prima dello scarto.

2. Contestazione di UNO:

- Se un giocatore rimane con 1 sola carta senza aver dichiarato preventivamente UNO, gli avversari possono premere il pulsante *"CONTESTA UNO"*;
- In caso di contestazione valida, il giocatore sbadato riceve **2 carte di penalità** dal mazzo;
- I Bot valutano automaticamente la contestazione in base alla propria percentuale di probabilità.

3.4 Gestione della Challenge sul Wild Draw Four (+4)

La carta +4 può essere giocata regolarmente solo se il giocatore non possiede altre carte del colore attivo. Quando viene giocata:

1. Il giocatore bersaglio riceve una richiesta di conferma per scegliere se **accettare le 4 carte** o lanciare una **sfida (Challenge)**.
2. **Risoluzione:**
 - Se la giocata era **illegale** (chi ha giocato il +4 possedeva il colore attivo), l'attaccante pesca le **4 carte** di punizione;
 - Se la giocata era **legale**, il bersaglio che ha contestato a vuoto pesca le **4 carte** originarie più **2 carte aggiuntive** di penalità e salta il turno.

3.5 La carta SKIP e lo Stacking

- Il pulsante *"SALTA"* è abilitato solo dopo aver pescato almeno una carta dal mazzo nel turno corrente (tracciato tramite `haPescatoNelTurno`);
- Durante una catena attiva di penalità nella variante *Stacking*, il passaggio del turno non è consentito: il giocatore deve rispondere con un'altra carta cumulabile (+2 o +4) oppure pescare l'intero ammontare accumulato.

4 Architettura del software (MVC)

Il progetto segue il pattern architetturale **Model-View-Controller (MVC)**:

- **Model (Package backend)**: Gestisce lo stato e la logica di dominio (carte, mazzo, gerarchia dei giocatori, calcolo punti). Non ha alcuna dipendenza da classi grafiche.

- **View (Package gui):** I pannelli Swing responsabili della visualizzazione dello stato di gioco e della cattura degli eventi dell'utente.
- **Controller (FinestraPrincipale):** Riceve gli eventi generati dalla View, applica le regole del Model, coordina i timer asincroni per i turni dei Bot e comanda l'aggiornamento grafico.

5 Diagramma UML delle classi

5.1 Struttura e relazioni del diagramma

In Figura 1 sono illustrate le classi principali del sistema e le loro relazioni:

- **Gerarchia dei Giocatori:** La classe astratta `Giocatore` definisce lo scheletro comune, specializzato poi da `GiocatoreUmano`, `BotCasuale` e `BotAggressivo`.
- **Modello delle Carte:** La classe `Carta` è composta dalle enum `Colore` e `Valore`. Ciascun giocatore gestisce la propria mano tramite una `List<Carta>`.
- **Controller:** `FinestraPrincipale` mantiene i riferimenti a tutti i sotto-pannelli del tavolo e gestisce il flusso delle schermate di dialogo.

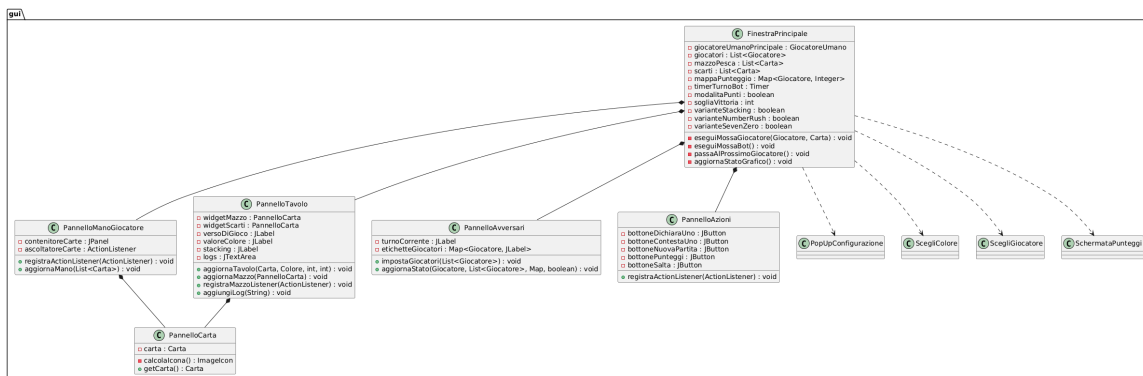


Figura 1: Diagramma UML delle classi principali.

6 Descrizione delle responsabilità delle classi principali

6.1 Package backend

- **Carta, Colore, Valore:** Definiscono i componenti base e i valori in punti del mazzo.
- **Mazzo:** Si occupa della generazione delle 108 carte regolamentari e del loro rimescolamento.
- **Giocatore:** Classe base astratta per i partecipanti, implementa `Comparable` e fornisce metodi di supporto per la gestione della mano e dei punteggi.

- **GiocatoreUmano**: Rappresenta il giocatore umano, delegando le scelte alle azioni dell'interfaccia grafica.
- **BotCasuale**: Bot semplice che seleziona la prima carta valida disponibile ed effettua scelte probabilistiche al 50%.
- **BotAggressivo**: Bot avanzato che privilegia le carte speciali per ostacolare gli avversari con poche carte, seleziona il colore più frequente nella mano e contesta le infrazioni con probabilità del 70%.

6.2 Package gui

- **FinestraPrincipale**: Classe principale (JFrame) che orchestra i turni e collega grafica e logica.
- **PannelloTavolo**: Area centrale con mazzo, scarti, indicatore colore/verso e riquadro dei log di gioco.
- **PannelloManoGiocatore**: Pannello inferiore con barra a scorrimento per visualizzare e selezionare le carte in mano.
- **PannelloAvversari**: Pannello superiore con riassunto visivo del turno e del numero di carte degli altri giocatori.
- **PannelloAzioni**: Colonna laterale con i pulsanti di comando (Dichiara/Contesta UNO, Salta, Nuova Partita, Punteggi).
- **PopUpConfigurazione**: Dialog modale per impostare partecipanti, modalità e varianti prima dell'avvio.
- **SchermataPunteggi**: Dialog con tabella ordinata per la visualizzazione della classifica dei punteggi.
- **StileUI**: Classe di utilità per il caricamento centralizzato del font *Fredoka*.

7 Costrutti del linguaggio Java impiegati

Nello sviluppo sono stati impiegati diversi costrutti della programmazione ad oggetti:

7.1 Interfacce

- **Comparable<Giocatore>**: Utilizzata per consentire l'ordinamento naturale dei giocatori.
- **ActionListener**: Impiegata nella GUI per la gestione asincrona degli eventi generati da pulsanti, carte e timer di gioco.

7.2 Generics e Collezioni

- **List<Carta>** (in particolare **ArrayList**): Utilizzata per la gestione dinamica del mazzo di pesca, gli scarti e delle mani dei giocatori.
- **Map<Giocatore, Integer>**: Utilizzata per memorizzare i punteggi progressivi dei partecipanti durante i tornei a punti.

7.3 Stream API ed Espressioni Lambda

- **Espressioni Lambda**: Utilizzate per definire in modo compatto i listener Swing e i timer dei turni bot.
- **Stream API**: Impiegate per filtrare e verificare la presenza di mosse valide nella mano (es. `mano.stream().anyMatch(...)`) senza ricorrere a cicli espliciti.
- **Comparator per la Classifica**: In **SchermataPunteggi** viene utilizzato lo stream ordinato con `.sorted((giocatore1, giocatore2) -> ...)` per generare la classifica decrescente.

8 Descrizione dell'interfaccia grafica con Screenshot

8.1 Schermata di configurazione iniziale

All'avvio compare la finestra di dialogo illustrata in Figura 2. Tramite questa schermata è possibile impostare:

- La modalità di gioco (*Partita Singola* o *Partita a Punti* con soglia regolabile);
- L'attivazione delle varianti opzionali (*Stacking*, *Number Rush*, *Seven-Zero*);
- Il numero di partecipanti, con la possibilità di assegnare a ciascuno un nome personalizzato e il relativo tipo (*Umano*, *Bot Casuale* o *Bot Aggressivo*).

Configura la Partita

Modalità di gioco

☒ Partita Singola ☐ Partita a Punti Soglia vittoria: 500

Varianti di Gioco Attive

☐ Stacking ☐ NumberRush ☐ SevenZero

Numero giocatori

2 3 4 5 6

Elenco Giocatori

Giocatore 1:

Giocatore 2:

Figura 2: Schermata per la configurazione della partita.

8.2 Tavolo da gioco principale

Durante la partita (Figura 3), l'interfaccia organizza lo spazio in quattro sezioni principali:

- **Area Superiore (PannelloAvversari)**: mostra il giocatore di turno e il numero di carte (e punteggio) di tutti i partecipanti;
- **Area Centrale (PannelloTavolo)**: ospita il mazzo di pesca cliccabile, gli scarti, l'indicatore del colore attivo, il verso di gioco e il riquadro scorrevole con i log degli eventi;
- **Colonna Laterale (PannelloAzioni)**: raggruppa i pulsanti per dichiarare o contestare UNO, visualizzare i punteggi, avviare una nuova partita e passare il turno con il pulsante *SALTA*;
- **Area Inferiore (PannelloManoGiocatore)**: barra a scorrimento orizzontale che visualizza le carte in mano al giocatore umano.



Figura 3: Schermata del tavolo da gioco.

8.3 Finestre di dialogo per i punteggi e le scelte

Al termine di ciascun round o su richiesta dell'utente tramite il pulsante dedicato, viene visualizzata la finestra di dialogo **SchermataPunteggi** (Figura 4), contenente una tabella ordinata che riassume i punteggi aggiornati in ordine decrescente.



Figura 4: Schermata punteggi e della classifica.

9 Descrizione delle Regole Alternative

Nel menu di configurazione iniziale è possibile attivare tre varianti di gioco opzionali, utilizzabili singolarmente o in combinazione.

9.1 Stacking (Accumulo +2 e +4)

Quando un partecipante subisce una carta penalità (+2 o +4), non è costretto a pescare immediatamente se possiede a sua volta una carta di penalità compatibile: giocandola, trasferisce la penalità accumulata al giocatore successivo. L'accumulo continua finché un partecipante non ha contromisure ed è costretto a pescare l'intero totale di carte accumulate (visualizzato in tempo reale sul tavolo da gioco).

9.2 Number Rush

Questa variante permette di scartare più carte dello stesso valore numerico nello stesso turno. Se un giocatore possiede, ad esempio, due carte con valore CINQUE (una Rossa e una Gialla) e la prima è giocabile sugli scarti, può scartarle entrambe contemporaneamente in un'unica mossa. Il colore dell'ultima carta scartata determina il nuovo colore attivo sul tavolo.

9.3 Seven-Zero

Introduce effetti speciali legati alle carte numeriche ZERO e SETTE:

- **Regola dello 0:** quando viene scartato uno ZERO, tutti i partecipanti cedono l'intera propria mano di carte al giocatore successivo, seguendo il verso di rotazione corrente.
- **Regola del 7:** quando viene scartato un SETTE, il giocatore di turno ha la facoltà di scegliere un avversario tramite una finestra grafica (`ScegliGiocatore`) e scambiare interamente le carte con lui.

10 Strategie dei Bot

I bot estendono la classe astratta `Giocatore` e interagiscono con il controller rispettando le medesime regole dei giocatori umani, adottando logiche decisionali differenti.

10.1 Bot Casuale

Il `BotCasuale` implementa un comportamento di base:

- **Selezione della mossa:** gioca la prima carta valida individuata nella propria mano;
- **Scelta del colore:** seleziona casualmente uno dei colori presenti tra le proprie carte;
- **Scambio del 7:** sceglie un avversario a caso con cui effettuare lo scambio;
- **Dichiarazioni e sfide:** risolve la dichiarazione di UNO, la contestazione e la sfida sul +4 con una probabilità fissa del 50% (testa o croce).

10.2 Bot Aggressivo

Il BotAggressivo adotta una strategia mirata a massimizzare l'intralcio agli avversari:

- **Priorità alle carte azione:** dà precedenza alle carte speciali (+4, +2, Salta, Inverti) per ostacolare gli avversari con poche carte;
- **Scelta del colore ottimale:** calcola la distribuzione dei colori nella propria mano e sceglie quello maggiormente presente;
- **Scambio mirato del 7:** individua e seleziona l'avversario con il minor numero di carte per massimizzare il proprio vantaggio;
- **Pressione su UNO e Challenge:** dichiara e contesta le dimenticanze di UNO con una probabilità del 70%. Inoltre accetta la sfida sul +4 se l'avversario ha meno di 3 carte o con il 70% di probabilità.

Tabella 2: Confronto decisionale tra Bot Casuale e Bot Aggressivo.

Decisione	Bot Casuale	Bot Aggressivo
Criterio di scarto	Prima carta valida	Priorità a carte speciali e +2/+4
Scelta del colore (Jolly)	Colore casuale	Colore più frequente in mano
Scambio carte (Regola 7)	Avversario casuale	Avversario con meno carte
Dichiarazione / Contesta	Probabilità 50%	Probabilità 70%
Challenge su +4	Probabilità 50%	Valutazione carte residue avversario

11 Istruzioni di compilazione ed esecuzione

11.1 Requisiti di sistema

- **Java:** JDK versione 21 o superiore;
- **Struttura cartelle:** è necessario mantenere i percorsi relativi predefiniti per la cartella `gui/images/` e per il font `gui/Fredoka.ttf`;
- **Risoluzione minima:** 1280×720 pixel.

11.2 Compilazione ed esecuzione da riga di comando

Dalla cartella radice del progetto:

- **Compilazione:**
`javac -d bin backend/*.java gui/*.java`
- **Avvio della GUI:**
`java -cp bin gui.FinestraPrincipale`

11.3 Esecuzione tramite IDE

1. Importare la cartella del progetto `unoProject` nell'IDE prescelto;
2. Selezionare il JDK 21 (o superiore);
3. Impostare come *Working Directory* la cartella radice del progetto;
4. Eseguire come applicazione Java la classe `gui.FinestraPrincipale`.